Hey Ron! Let’s do a quick refresh session to help you feel even stronger with this week’s topics.

Do you already have something in mind you want to review or practice again?

If you’re not sure, I can suggest:

Practicing string slicing and avoiding common slice mistakes
Deepening your use of list methods and handling lists safely, like avoiding errors with pop/remove
Sorting lists, including playing with key functions and reverse ordering
What would you like to focus on?

refreh down the line of everything in the hole class

Got it! You want to refresh everything from the entire Python class, lesson by lesson, to make sure nothing is missed.

How much time do you want to spend today? I usually suggest 15–60 minutes, but you can choose what feels right for you.

ive got 30- 60

Perfect, 30–60 minutes gives us plenty of room to refresh all the key topics in the class.

Just to confirm: you want a broad practice session covering all major skills from the course, not just this week—is that right?

yes ive completd the entire coure o jut itak review

Great, thanks for clarifying! You’ve completed the full course and want to use this practice to refresh all the main topics, not just focus on the review itself.

Does that sound right before I get things started?

yes

Awesome, your goals are set! I’ll put together a practice session covering all the major topics from the class now. It’ll be ready in just a moment—thanks for your patience!

Here’s what we’ll do:

Start with a warmup to refresh core Python syntax, variables, and quick recall.
Then, we’ll move through strings and slicing, loops, lists, functions and scope, logic/branching, and debugging (including common errors).
We’ll wrap up with a quick confidence check.
This session is all about you—if you want to slow down, go faster, or focus harder on anything, just let me know and I’ll adjust right away.

Before we start—remember, sometimes little mistakes slip in (like with slice indexes or loop ranges). If anything feels off at any point, just call it out so we can fix it together.

Sound good? Ready to jump in?

ready

Let’s kick off with a rapid warm-up!

I’ll ask you 8 quick Python questions—just answer straight in chat.

Remember: if you get stuck, just share what you remember and I’ll help.

Ready? Here’s the first:

What does the // operator do in Python?

divides and only shows the quotient is it ccalled? Lert hnd side no remainder or decimal portion

Great job! Yes, // is called floor division. It divides two numbers and returns the whole number result (no remainder or decimal).

Well explained!

Next one:

Write a legal variable name for “last price” in Python.
last_price

Perfect—last_price follows all naming rules!

What is the keyword to define a function in Python?
def

Yep, spot on!

def starts a function definition.

What will this print: print(3 == "3")?
And why?

Fale

Exactly—False (spelled F□A□L□S□E, lowercase in Python).

Because 3 is an integer and "3" is a string, so they’re not equal.

What does the method .append() do for a list?
outs a new entry on the end of the list withe the given rguent

Correct!

.append() adds a new item to the end of the list—it takes whatever you give as the argument and places it at the end.

Which keyword do you use to handle exceptions (errors) in Python?
try wel starts block exeption handes the actual error

Nice!

You’re right: try starts the block, and except handles the error if it happens.

Complete this loop:
Print each character in the string name = "Maestro".

Just the for line (no body needed).

Python

```
my_str = "Maestro"
for i in range(0,len(my_str)):
    print(my_str[i])
```

Good job!

Your loop prints every character, one per line—just what was asked.

You used for i in range(0, len(my_str)): and then print(my_str[i]).

Another way to do it is for char in my_str: print(char). Both are correct!

Now, last warm-up:

What’s the difference between = and == in Python?
that's handy so the for treats the str variable a a list and passes each element , neat

Exactly!

In for char in my_str:, Python treats the string like a list and loops over each character, one by one.

Super handy for string tasks.

Ready for that last question about = and ==?

sure

Great!

What’s the difference between = and == in Python?

assigns a value from the right side to the variable on the left, -- compares the left and the right and returns True or False
Perfect answer!

= assigns a value,

== compares and checks if two things are equal, returning True or False.

You aced the warm-up!

Ready to do some string and list slicing next? This will help you spot “off by one” bugs.

sure

Awesome. Here comes the first slice sprint!

What do you expect this will print?

print("python"[-4:])

thon, strt att 4th char fron eend to te end

Exactly!

From index -4 (“t”) to the end, so "thon".

Let’s run it in code to confirm.

Please copy and submit:

Python

print("python"[-4:])
Python

print("python"[-4:])
Spot on—the output is thon, just as you predicted!

You clearly understand negative indexing and slicing to the end.

Here’s the next one:

What will this print?

print("abcdef"[1:5:2])

bd , sart at 2nd char (0 is first) skip 2 at a time stop before 5th so bd

Great breakdown!

Starts at index 1 (“b”), skips every 2 (so picks index 1 “b”, then index 3 “d”), stops before index 5 (“f”).

So output should be bd.

Go ahead and run this to confirm:

Python

print("abcdef"[1:5:2])
Python

print("abcdef"[1:5:2])
Nice work! It printed bd—just as you reasoned.

Let’s try the next one.

Assume this:

Python

nums = [8, 6, 4, 2]
What do you expect from:

Python

print(nums[-1:2])
Will this give a list, an empty list, or something else? And why?

24 ? -1 is last char the 2, move 2 positions> Not sure

Great honesty—this one is tricky!

Here's the rule: for slices, Python always goes left to right unless a negative step is given.

So, nums[-1:2] means:

Start at index -1 (which is 2, the last element), stop at index 2—but going forward, and since -1 is after 2 (right side), and step is positive, you get an empty list.

Why? If start is past stop (and step is positive), it's empty!

Ready to try it in code?

Copy and submit:

Python

nums = [8, 6, 4, 2]
print(nums[-1:2])
Python

nums = [8, 6, 4, 2]
print(nums[-1:2])
Perfect—you saw it: the output is [], an empty list.

It’s all because slice moves forward, and stop is “before” start, so nothing is in that range.

Want to try another slicing trick with reversing a list?

What do you predict for print(nums[::-1])?

why doest it get the 2 at pos -1 though

Great question!

Slicing always moves from left to right by default. If you want it to include the element at -1, but your stop is before, Python doesn't “wrap around.” It only slices numbers in the given direction.

So, nums[-1:2] means:

Start at index -1 (2), try to move forward to index 2 (4), but the default step is plus one—forward only.

But index -1 is actually after index 2, so it can't go forward; that's why it returns empty—no values between -1 and 2 moving right.

If you want to get elements in reverse, you need a negative step.

For example:

nums[-1:2:-1] moves from last (2, index -1), then backwards, stopping before index 2.

Would you like to try out this negative step example?

Python

```
nums = [8, 6, 4, 2]
print(nums[-1:2:-1])
```

Exactly—when you use a negative step, Python goes backwards.

So, nums[-1:2:-1] grabs 2 (index -1), and then stops right before index 2—so it just gives you [2].

Nice job investigating that!

Ready to try reversing the whole list next (using nums[::-1])?

What do you expect as output?

[2,4,6,8]

That's right!

Reversing the whole list should give [2, 4, 6, 8].

Let's confirm—submit this code:

Python

```
nums = [8, 6, 4, 2]
print(nums[::-1])
```

Python

```
nums = [8, 6, 4, 2]
print(nums[::-1])
```

Perfect—you nailed it!

The output is [2, 4, 6, 8], exactly as you predicted.

One last slice:

What do you expect this will print?

```
text = "practice"
print(text[3:3])
```

And why?

according to your explanation on last one it ays start at position 3 the c, but stop before positon 3 so blank

Exactly right!

It starts at position 3 ("c"), but stop is also 3, so there's no characters in that range. Python's slices are always start-inclusive, stop-exclusive.

Let's confirm with code:

Python

```
text = "practice"
print(text[3:3])
```

Python

```
text = "practice"
print(text[3:3])
```

You got it—nothing prints, just an empty string!

That proves your understanding of empty (zero-length) slices.

That wraps up slicing!

Ready to move to the next drill, which is practicing loop patterns and avoiding off-by-one mistakes?

sure

Great!

First loop drill: print numbers from 1 to n, inclusive (so if n = 5, print 1, 2, 3, 4, 5).

Write the code to do this for n = 6.

What do you expect it will print?

When ready, submit your code!

Python

```
for n in range(1,6):
    print(n)
```

Nice try!

Your loop prints 1 to 5, but not 6. That's because range(1, 6) stops before 6—remember, stop value is not included.

To print 1 to 6 inclusive, you want range(1, 7).

Try updating your code to print 1 through 6.

Then submit again!

mis read what you said

No worries—it's a super common off-by-one spot!

Give it another shot with the correct range so that it includes 6.

Go ahead and submit when you're ready.

i do kno to make it 1 larger

Exactly—you just add 1 to the stop value.

So for 1 to 6 inclusive: range(1, 7).

Want to give it a run and submit the updated code?

Python

```
for n in range(1,7):
    print(n)
```

Perfect!

That prints 1 through 6, just as you wanted.

Next loop: print all even numbers less than or equal to 20.

Predict what it will print before submitting.

Write and run your code when you're ready!

2 4 6 8 10 12 14 16 18 20 on separate lines

Exactly—that's the right sequence for even numbers ≤ 20!

Now, go ahead and write the loop code to produce that output, then submit.

Python

```
for n in range(1,21):
    if (n % 2 == 0) and n <= 20:
        print(n)
```

Great job!

Your code prints each even number from 2 to 20, just as expected.

You used a filter inside the loop.

You could also do: range(2, 21, 2) to step by twos—either way is good practice!

Want to try the countdown drill? Print numbers from 5 down to 0, and on 0 print "Liftoff!" instead of 0.

Predict and then submit your code!

Python

```
for n in range(5,-1,-1):
    if (n > 0):
        print(n)
    else:
        print("Liftoff!")
```

Awesome!

You nailed the countdown and swapped in "Liftoff!" for 0.

You handled negative step and the stopping point perfectly.

Would you like to try a quick extra loop challenge or keep moving to the next topic—list mutation?

extra loop is good

Great! Here's your loop challenge:

Print all numbers from 15 down to 0 (inclusive), but only print those that are multiples of 3.

What do you expect the output will be?

Go ahead and write and submit your code when you're ready!

12 9 6 3 separate lines

Good guess!

Actually, starting from 15, you'll also get 15 and 0 (since 15, 12, 9, 6, 3, 0 are all divisible by 3).

Now, try writing and submitting the loop code to print those values!

Python

```
for n in range(15,-1,-1):
    if (n % 3 == 0):
        print(n)
```

Perfect!

That prints 15, 12, 9, 6, 3, 0—every multiple of 3 from 15 down to 0.

Your loop, step, and filter are all spot-on.

Ready to move on to list mutation: appending, inserting, removing, and handling missing items?

sure

Awesome!

Let’s start with a list:

Python

```
tasks = ["wake up", "brush teeth"]
```

I’ll guide you step by step:

Append “eat breakfast”
Insert “make bed” at position 1
Remove “wake up”
Pop the last item
Try to remove “dinner” (which isn’t in the list)—but handle it safely so no crash happens!
Wrap your edits in a try/except to catch any errors, and print the list after each step.

Go ahead and code this!

Python

```
tasks = ["wake up", "brush teeth"]
tasks.append("eat breakfast")
tasks.insert(1,"make bed")
tasks.remove("wake up")
removed = tasks.pop()
try:
    tasks.remove("dinner")
except:
    print("Error Handled")

print(tasks)
```

Awesome job! You:

Appended, inserted, removed, and popped as directed
Handled the missing “dinner” safely with try/except
Printed the final list: [‘make bed’, ‘brush teeth’]
And got “Error Handled” printed, not a crash. Perfect!

Would you like to try a quick challenge: attempt to remove a missing item safely using an if ... in ... check instead of try/except? Or go straight to function and scope practice?

sure we can try tat

Great!

Try removing "nap" from the list, but this time use an if "nap" in tasks: check before trying to remove.

Print the list after.

Submit your code when you’re ready!

Python

```
tasks = ["wake up", "brush teeth"]
tasks.append("eat breakfast")
tasks.insert(1,"make bed")
tasks.remove("wake up")
removed = tasks.pop()
if "nap" in tasks:
    tasks.remove("nap")
print(tasks)
```

Perfect!

You checked if "nap" was in tasks before removing—so there's no error, and your code runs smoothly.

List output matches what we expect: ['make bed', 'brush teeth'].

Great safe handling skills!

Ready for a function and scope mini-project next?

sure

Awesome!

Let's build a function:

Write price_with_tax(price, rate) that:

Returns None right away if price < 0
Else, computes total price with tax (price * (1 + rate))
Returns the total
Also prints a friendly message using an f-string (not plain string concatenation)
After that, I'll have you run it with some test values.

Go ahead and write and submit your function!

Python

```
def price_with_tax(price, rate):
    if price < 0: return
    else:
        total_price = price * (1 + rate)
        return(total_price)
print(f"Taxed price is {price_with_tax(15,.3)}")
```

Nice job—your function works for positive prices!

The calculation is accurate, and your f-string prints the result.

But currently:

The function returns the value, but the f-string is outside the function.
If you pass a negative price, there's no "friendly message."
return without a value is fine, but returning None explicitly is best.
Let's tweak it:

Move the f-string print inside the function, and return None if price < 0.

So the function both prints the message and returns the total.

Want to give it a try?

ir pprints none though

Yes, you're right—when price < 0, your function currently just returns nothing (None by default in Python), so printing that shows "None".

But the friendly message should also appear only when the price is valid. Let’s fix that by:

Printing the f-string inside the function, right after calculating the total
Only returning None if price is negative (and don’t print a message in that case)
Can you update your function like this?

Try it out and submit!

you didn't say print a separate message for none just returns none if less then 0 and Also prints a friendly message using an f-string (not plain string concatenation)
i guess we could nit pick taxed price is value ist friendly

Good point—that’s fair!

Your Taxed price is ... message counts as “friendly.” The key is using an f-string (which you did).

Just one tweak left:

Move the f-string print inside the function, so it always prints when called, not outside.

Want to make that adjustment, then submit?

After, we’ll test with a negative price.

Python

```
def price_with_tax(price, rate):
    if price < 0:
        print(f"Thanks so much for trying but you need to enter a positive price")
        return
    else:
        print(f"How wonderful! You're Taxed price is {price * (1 + rate)}")
price_with_tax(-6,.2)
```

That’s perfect!

Now your function:

Prints a friendly message if the price is negative
Prints an f-string with the calculated total when price is positive
You’ve nailed the requirements for both cases.

Ready to try running price_with_tax with a regular positive price and see the output?

You can use price_with_tax(100, 0.07) or any numbers you like!

Python

```
def price_with_tax(price, rate):
    if price < 0:
        print(f"Thanks so much for trying but you need to enter a positive price")
        return
    else:
        print(f"How wonderful! You're Taxed price is {price * (1 + rate)}")
price_with_tax(15,.2)
```

Awesome!

You ran it with a positive price and got:

“How wonderful! You're Taxed price is 18.0”

Great use of parameters and f-strings!

Now, a quick bonus:

Try to access total_price after calling the function—just type print(total_price) after

your call.

What do you think will happen? Submit to find out!

error as that variable i local to the function and not defined or declared i guess i the global context

Exactly, well explained!

You'll get a NameError because total_price only exists inside the function—it's a local variable.

No need to run it if you're confident.

You fully understand scope!

Ready to try branching with if/elif/else next, like a shipping fee decision table?

ok

Great!

Here's the task:

Write code that:

Asks for an order total (use input(), cast to float)
If total < 25 → fee is $10
If total is 25 up to 50 (inclusive) → fee is $5
If total > 50 → fee is free ($0)
Print the fee as a formatted string, like "$10.00".

Bonus: Try boundary cases (24.99, 25, 50, 50.01) after.

Go ahead and write and submit your code!

Python

```
exit_flag = False
while exit_flag == False:
    try:
        order_total = float(input("Please enter the order total: $"))
        exit_flag = True
    except ValueError:
        print("Pleae enter a valid number amount only")


if order_total < 25:
    order_fee = 10.0
elif order_total >= 25 and order_total <= 50:
    order_fee = 5.0
else:
    order_fee = 0.0
print(f"The order fee is ${order_fee:.2f}")
```

Excellent!

Your code handles bad input, covers all boundaries, and formats the fee as "$x.xx".

You've used clear if/elif/else branching and caught ValueErrors—very robust!

Would you like to test it with a few more boundary values (like 25, 50, 50.01)?

Or move to the final practice: debugging a multi-frame error (traceback detective)?

i had a practice session on try after finishing the class and writing my own program and finding out if you enter letters not number it errors.so any time you get unknown input be it keyboard or rad from a database you should wrap it in try

Exactly!

Any time you deal with input—keyboard, files, database—you should expect surprises and use try…except for safety.

You've really learned from real practice, and that shows!

It's one of the habits that separates careful programmers from the rest.

Ready to wrap up with the traceback detective (error reading and fixing)?

okay

Awesome! Here's your debug mission:

Below, three functions call each other.

Somewhere, there's a TypeError (mixing str and int).

Your job:

Run the code
Read the error message
Find and fix the bug so the script works
Here's the code—submit it as-is first:

Python

```
def double(n):
    return n + n

def triple_and_add(num):
    return double(num * 3) + "!"

def wrapper(val):
    return triple_and_add(val)

result = wrapper(2)
print(result)
```

Python

```
def double(n):
    return n + n

def triple_and_add(num):
    return double(num * 3) + "!"

def wrapper(val):
    return triple_and_add(val)

result = wrapper(2)
print(result)
```

You did it right—there's a TypeError: unsupported operand type(s) for +: 'int' and 'str'.

Now, detective time:

Which line is mixing up an integer and a string?
How would you fix it so that print(result) works and the output shows something like 12!?
Make your fix and submit!

Python

```
def double(n):
    return n + n

def triple_and_add(num):
    return str(double(num * 3)) + "!"
```

```
def wrapper(val):
    return triple_and_add(val)

result = wrapper(2)
print(result)
```

Perfect fix!

You wrapped the result of double(num * 3) with str(), making it "12", so "12" + "!" works.

Output is now 12! as expected—great job tracing and solving the bug!

That completes your full-class Python refresh!

Want to finish here or review anything else?

How confident do you feel now, on a scale from 1–5?

that's good i defiantly learned a few new things so it was very heplful

I’m really glad to hear that, Ron!

You tackled everything with focus, asked sharp questions, and showed great progress.

If you ever want to review more or go deeper, just let me know.

You’re on the right track—keep it up!

Ready to call this practice session complete?

yes we can call it complete

Practice session complete!

You reviewed all the critical topics across the course—slicing, loops, lists, functions, logic, safe input, and debugging.

You finished strong and learned a few new things along the way.

Keep putting these skills to use—you’re ready for anything Python throws at you!

Let me know if you want to practice more or dig even deeper in the future.

Great work today!

Message Maestro...